

1. 项目背景与目标

本项目旨在构建一个模块化的全屋智能中控系统。核心是通过语音识别（ASR）与自然语言处理（NLP）技术，打破传统 App 手动控制的繁琐，实现“即说即得”的交互体验。

2. 需求分析 (Requirement Analysis)

2.1 功能性需求

- 语音处理模块：**支持实时唤醒、语音转文字（ASR）、意图解析（NLU）及语音合成（TTS）。
- 设备控制模块：**支持灯光（开关/亮度）、空调（温度/模式）、安防（摄像头/门禁）的逻辑控制。
- 环境感知模块：**实时查询天气、室内温湿度数据。
- 多端交互：**提供移动端 App 与 Web 控制台。

2.2 非功能性需求

- 低延迟：**语音指令从输入到执行的端到端延迟控制在 1.5s 以内。
- 鲁棒性：**具备回声消除（AEC）与背景降噪能力。
- 可扩展性：**采用插件式架构，支持通过配置文件接入新协议设备。

3. 技术栈选型 (Technical Stack)

维度	推荐方案	说明
开发语言	Python 3.10+ / C++ 20	Python 负责 AI 与业务逻辑，C++ 负责底层设备驱动模拟。
API 框架	FastAPI	高性能异步框架，支持自动生成 OpenAPI (Swagger) 文档。
网络通信	MQTT + WebSocket	MQTT 负责物联网设备低功耗长连接，WebSocket 负责 App 实时状态推。
服务端框架	Docker + Redis	使用 Docker 进行容器化部署，Redis 存储设备影子（Shadow）状态。
移动端框架	Flutter / React Native	一套代码双端运行，快速构建交互 UI。
AI 推理引擎	Whisper / Qwen / FunASR	采用预训练模型实现高精度的 ASR 与 NLP。

4. 系统架构设计

系统采用分层架构：

- 感知交互层：**麦克风阵列拾音、App UI。
- 核心服务层：**ASR/NLP 引擎、业务路由、用户鉴权。
- 抽象驱动层：**将 Zigbee, Matter, WiFi 设备统一抽象为标准对象。
- 硬件/仿真层：**实际智能硬件或基于 C++ 编写的设备仿真器。

5. 开发计划 (Development Roadmap)

采用 4 个 Sprint (冲刺), 每个周期 1-2 周。

Sprint 1: 骨架搭建与协议定义

- 任务: 定义 MQTT 通信协议规范; 搭建基于 FastAPI 的后端基础镜像。
- 交付物: API 接口文档 (Swagger)、核心数据库 Schema。

Sprint 2: 语音与 AI 模块集成

- 任务: 集成 ASR 与 TTS 服务; 实现基于大模型 (LLM) 或语义槽匹配的意图解析。
- 交付物: 语音识别 Demo, 能够将语音解析为标准 JSON 指令。

Sprint 3: 业务逻辑与 App 开发

- 任务: 实现设备管理、场景联动 (如“回家模式”); 同步开发 Flutter 移动端。
- 交付物: 可操控的 App 原型。

Sprint 4: 性能优化与交付

- 任务: 抗干扰测试、压力测试; 完善部署文档与用户手册。
- 交付物: 全套源码、Docker Compose 一键部署包。

6. 任务分配 (Team Allocation)

针对 6 人团队的职能划分:

角色	人数	主要职责
Scrum Master / PM	1	需求把控、文档统筹、Sprint 进度协调。
AI & 算法工程师	1	负责 ASR/NLP 模型的部署、微调及语音处理流水线。
后端工程师 (Core)	2	负责 FastAPI 业务逻辑、MQTT 消息代理、数据库设计。
前端/移动端工程师	1	负责 Flutter App 开发与 Web 管理后台。
系统/嵌入式工程师	2	负责设备驱动抽象、C++ 仿真器编写、网络拓扑维护。

7. 测试框架 (Testing Framework)

为了保证模块化开发的质量, 必须坚持测试驱动 (TDD) 理念:

- 单元测试 (Unit Test):
 - Backend: 使用 `pytest` 对业务函数进行 100% 覆盖。
 - Device Logic: 使用 `Google Test (GTest)` 测试 C++ 编写的驱动逻辑。
- 集成测试 (Integration Test):

- 模拟语音指令 -> ASR -> NLP -> MQTT 指令下发，验证全链路闭环。
- **压力测试 (Load Test):**
- 使用 **Locust** 模拟多设备并发上报状态，测试后端的并发处理能力。
- **UI/自动化测试:**
- 使用 Flutter 的集成测试模块验证关键操作路径。